

Airbnb Price Prediction - Barcelona

Machine Learning Project - SoSe 2026

Nur Azreik Victor Willemer Trung Pham

Georg-August-Universität Göttingen

July 15-17, 2026

Step 0: Data Import

- We imported two datasets (`listings.csv` and `reviews.csv`) even though we work with the reviews.
- We do that because there is a mismatch between `listing_id` that is in `reviews.csv` and `id` that is in `listings.csv`.
- Both are the same but in each file their names are different.

Step 1: Data Cleaning and Formatting

- Once the data was mapped, we checked for duplicate columns and dropped them.
- Then we removed unnecessary columns such as “reviewer_name” and “id” (comment ID).
- Then we filled missing data (NaN comments), cleaned line breaks, forced all comments into strings, and cleaned up accidental extra spaces in the beginning/end of each file.

Step 2: Data Consistency and Overlap

- To ensure that our model learns from actionable data, we compared the unique property IDs across both datasets.
- We isolated and kept only the reviews that successfully mapped back to the core `listings.csv` file.

Step 3: Guest Behaviour

- We analyzed the reviewers to see how many reviews individual users were leaving.
- We did that to check if we could add a “reviewer weight” to the comments we will work with later. Spoiler: it didn't.
- The distribution was heavily skewed; the majority of the reviewers left one review but a small group of active reviewers left over 20 reviews.

Step 4: Reviewers Weight

- We wanted to get the amount of reviews a user left on a unique listing.
- We applied a log-scaled weight to this count (because of the skewness).

Step 5: Language Analysis

- Before using a pre-trained model, we had to check the language distribution of the comments.
- Sampled 5,000 comments and ran them through a language detection tool, filtering comments that are too short/unreadable to be classified.
- Confirmed that we must use a multilingual model.

Step 6: The Model

- We chose a BERT model that handles multiple languages, but also has a built-in scoring system, because it was trained extensively on customer feedback.
- The model classifies text into a 1-5 stars scoring system.

Step 7: Scoring Function (Formatting)

- The model itself gives out a classic 1-5 stars rating, but we wanted a more precise number for a precise prediction.
- We wrote a function that feeds the reviews to the model in small batches so the computer won't run out of memory.
- We disabled model training as well for faster GPU performance.
- The function formats the text to keep it all the same length. Because the text is transformed into tensors/matrices and comments are in different lengths, they do not all fit in the same size.
 - `padding = True` adds empty filler tokens, so that shorter comments will be filled to the max length.
 - `truncation = True` and `max_length = 512` act as a ceiling, chopping off comments that are longer than 512 tokens.
- Here I could use a chunking technique to fit all comments even if they were cut! But because only 0.26% are longer than 512 tokens, I didn't touch it.

Step 8: Scoring Function (Math)

- Instead of accepting the model's default rating (1-5 integer stars), the model calculated "softmax": softmax gives us the confidence percentage for all the 5 star levels.
- The model generates something called "logits" which represents how strongly the model feels about each rating.
- The softmax function assigns every single star a confidence percentage:
 - 1 star: 5% confident $\rightarrow 0.05$
 - 2 stars: 10% confident $\rightarrow 0.10$
 - 3 stars: 20% confident $\rightarrow 0.20$
 - 4 stars: 40% confident $\rightarrow 0.40$
 - 5 stars: 25% confident $\rightarrow 0.25$
- So it calculates: $0.05 \times 1 \text{ (stars)} + 0.10 \times 2 \text{ (stars)} + \dots$
- And adds them all together: $0.05 + 0.20 + 0.60 + 1.60 + 1.25 = \mathbf{3.70}$

Step 9: Feeding the Model

- We converted our entire column of clean text into a simple list format and fed the model (using the score function and the translation).

Step 10: Check

- Ran a quick check with top-scored comments and least-scored comments and validated that everything was correct.

Step 11: Applying Weights / Splitting Time

- Loaded the review scores and formatted the dates, to ensure that experienced guests' weight had a proper impact on the sentiment score.
- We found the most recent review in the dataset was exactly 12 months back (2024/09/05).
- Split the data into two groups: recent and older reviews, to understand if properties' reviews are improving/declining over time.

Step 12: Aggregating

- After every individual review had a score, we needed to group them all together by the specific property they belong to.
- For each listing, we calculated a summary of overall performance, finding:
 - Average median score
 - Best and worst reviews
 - Total number of reviews
 - Very first/very last reviews timeline
- And we get the weighted mean sentiment to get an overall score for each property that considers the experienced reviewers' weight.

Step 13: Reviews Consistency

- We want to understand the reviews' consistency over time.
- We calculate sentiment range by: $\text{max_score} - \text{min_score}$, which can tell how consistent the reviews are (is the quality swinging or is it a steady experience).
- Then we calculate properties that the scores are under 3 stars percentage and merge it into our data.

Step 14: Tracking Time and Range

- We calculate the days since the last review for the model, so that it knows if a property is still actively reviewed or not.
- We calculate review velocity (average number of reviews of a property each month). This could be a good indicator of how frequently a property gets booked since Airbnb doesn't provide that number.
- We wanted to know if a host's quality is improving or declining:
 - We isolate their average score for just the last 12 months and subtract the older historical average from it.
- This created a `sentiment_trend` feature that our model can use to tell if a reputation of a property is going up or down.

Step 15: Final Cleanup

- If a property didn't have recent reviews, we filled it with its overall average score.
- If a property didn't have enough data to calculate a trend or std, we set those values to 0 so the model wouldn't crash.
- We dropped temporary math columns, leaving only finalized data.

Step 15: Data Splitting

- We split our data into two sets so we could compare their effect on the training.
- Both sets share the exact same features. The only difference is:
 - The first dataset (**unweighted**) uses a standard average - every single reviewer is the same.
 - The second dataset (**weighted**) uses a custom weighted average where reviews from highly experienced reviewers are more considered.

Step 15: Validation & Step 16: Exporting

- **Correlation Check:**

- To make sure the custom weights didn't break the scoring system, we ran a mathematical check called "Pearson correlation," comparing the standard average against our new weighted averages.
- The correlation came back as **0.9954**.
- Which told us that the overall rating stayed mostly consistent, but the weights successfully applied a subtle, targeted shift based on guest experience. This means it slightly adjusted the rating.

- **Step 16: Exporting:**

- Exporting both weighted and unweighted files so we could compare both results.